

Algoritmos y Estructuras de Datos

Estructuras y su Sintaxis en C#



Contenidos del documento:

- ❖ Declaracion de variables
- ❖ Lectura y Escritura de datos por consola
- ❖ Condicionales If, Else, Else- If, Switch
- ❖ Ciclos For, While, Do-While
- ❖ Funciones

Declaracion de variables:

En otros documentos hablamos de la estructura que tienen las variables, bueno de la siguiente manera nosotros declaramos una variable.

```
// Con la doble barra nosotros podemos escribir codigo que no se va a
ejecutar, a esto se le llama comentar

// Declaramos en el siguiente orden 1- Tipo de dato 2- Nombre de la
variable 3-Asignacion de valor

int x = 1;

string a = "Hola";

bool b = false;

float c = 0.0f;

char t = 'a';


// Podemos declarar las variables sin un valor inicial.

int h;

string s;
```

Lectura y Escritura de datos:

Nosotros vamos a visualizar e ingresar datos a traves de la consola, para hacer eso vamos a utilizar las siguientes funciones.

```
// A lo que esta dentro de los parentesis de cada funcion se le llama
parametro

// Esta funcion escribe algo en la consola y luego 'apreta enter' osea que
hace un salto de linea

Console.WriteLine("Hola");

Console.WriteLine("Que tal");
```

```
// Esta funcion escribe algo pero no hace el salto de linea
Console.Write("Chau");

Console.Write("Chau");

//Para leer datos de la consola, camos a tener que guardarlos en una
variable

String x;

x = Console.ReadLine();

//Esto va a devolver lo que pongamos en la consola como un string

//el programa se va a detener hasta que mandemos algo

//Ahora si queremos guardar un int tenemos que convertirlo

int a;

a = Convert.ToInt32(Console.ReadLine());

// Convert.ToInt32 va a convertir el parametro a int antes de guardarlo
```

Condicionales IF, ELSE, ELSE IF:

Los condicionales nos permiten lograr que algunas líneas de código se ejecuten solamente bajo ciertas condiciones.

```
int a = 5;

int b = 6;

// podemos declarar directamente los valores pero referencia a la variable
es mas prolijo

//if es una funcion que recibe como parametro una operacion logica, si el
resultado de la operacion es verdadero

// se ejecuta el codigo de adentro, en caso contrario se ignora

// este codigo no se va a ejecutar

if (a == b) {

    Console.WriteLine("5 es igual a 6");

}

// si queremos que cuando la operacion logica no sea verdadera se ejecute
otra linea de codigo usamos Else
```

```
// esto va a imprimir en la consola "a es distinto de b"

if (b == a){

    Console.WriteLine("a es igual a b");

}else {

    Console.WriteLine("a es distinto de b");

}

// otra forma de correctamente estructurar estas linea de codigo

if (a == b) { Console.WriteLine("a es igual a b"); } else {
Console.WriteLine("a es distinto de b"); }

// si queremos que el else se ejecute bajo otra condicion usamos else if

if (b == a){

    Console.WriteLine("a es igual a b");

}else if (b < a){

    Console.WriteLine("b es menor a a");

}else {

    Console.WriteLine("b es mayor a a");

}
```

Switch:

```
// switch se usa para que una linea de codigo se ejecute cuando una
// variable tiene un valor especifico

int x = Convert.ToInt32(Console.ReadLine());

switch (x)
{
    case 1:
        Console.WriteLine("Seleccionaste el 1");
        break;

    case 2:
        Console.WriteLine("Seleccionaste el 2");
        break;

    case 0:
        Console.WriteLine("chau");
        break;

    default:
        Console.WriteLine("No seleccionaste ninguno de los numeros");
        break;
}
```

Ciclos For, While y Do-While:

```
// Los 3 argumento que lleva son, 1- donde empieza i 2- que condicion se
tiene que cumplir para que termine

// y cuanto vamos a sumar a i por iteracion

for (int i = 0; i < 10; i++)

{

    Console.WriteLine(i);

}

// el bucle while recibe como parametro una operacion, mientras esa
operacion sea verdadera se va a seguir ejecutando

int x = 3;

while(x==4){

    Console.WriteLine("Brasil");

    x = x + 1;

}

// el bucle do while funciona de la misma manera que while pero se ejecuta
si o si al menos una vez

int x = 3;

do

{

    x = x + 1;

    Console.WriteLine("Brasil");

} while (x == 4);
```

Funciones:

Todas las que venimos usando son funciones prearmadas por el lenguaje de programacion que estamos usando. Podemos crear nuestras propias funciones para resolver diferentes problemáticas que tengamos.

```
// Nuestras funciones van a tener varios atributos

// El nivel de acceso que tiene, osea quienes pueden llamarla

// a quien pertenece, static significa que pertenece al mismo
codigo donde estamos posicionados

// tipo de dato que va a devolver la funcion

// nombre de la funcion

// parametros que recibe

public static int sumar(int a, int b){

    int res = a + b;

    return res;

}

//nosotros podemos sumar de una manera mas simple, este es un
ejemplo de una funcion que suma

// si queremos usarla ahora tenemos que llamarla como a
cualquier otra funcion
```




```
// vamos a hacer que la variable x sea igual a lo que
devuelva la funcion sumar con los parametros 5 y 2

int x = sumar(5,2);

//se utiliza igual que otras funciones como por ejemplo
console readline

// Console.ReadLine especificamente no recibe parametros pero
por ejemplo WriteLine si lo hace

string y = Console.ReadLine();

Console.WriteLine(y);
```

■ ■ ■